

Instituto Tecnológico de Colima

Ingeniería en Inteligencia Artificial

Cómputo en la Nube

Proyecto Nebula Arena — Unidad 3

Guion de la presentación final

Jarom Cortes Perez

Carlos Antonio Vogel Romero

11 de noviembre de 2026

Introducción

El material pide una presentación de 10 minutos con una demostración en vivo del autoescalado. Este guion dice qué vamos a decir en cada bloque, cuánto dura y qué enseñamos en pantalla. Al terminar cada sesión destruimos el clúster de AWS, porque encendido cuesta unos USD 8 al día. Para la demo lo volvemos a crear con `terraform apply` (unos 15 minutos) antes de la clase y lo destruimos después.

Bloques de la presentación

Minuto	Bloque	Qué decimos y qué enseñamos
0:00–0:45	**El problema**	Un juego en línea tiene picos: 100 jugadores a las 3 de la tarde y 1,000 a las 9 de la noche. Pagar servidores para el pico todo el día es caro, y quedarse corto deja a los jugadores sin partida. Nuestro objetivo: que la plataforma crezca y se encoja sola, y saber cuánto aguanta y cuánto cuesta.
0:45–1:30	**El juego**	10 segundos de una partida 5 contra 5. "El juego es el caso de estudio; lo que evaluamos es la infraestructura."
1:30–3:00	**Arquitectura**	El diagrama del reporte: matchmaker, Agones y servidores de partida; Prometheus y Grafana; la tabla de líderes serverless; el pipeline a ECR; Terraform. Explicamos las tres capas de escalado: flota, pods y nodos.
3:00–6:00	**Demo en vivo**	Ver la sección siguiente.
6:00–7:30	**Resultados**	La rampa de 300 jugadores en EKS: la flota pasó de 3 a 37 servidores y los nodos de 4 a 6; máximo de 252 jugadores, unos 42 por nodo; la flota reaccionó en 43 segundos y cada nodo tardó ~1 minuto. Enseñamos la gráfica de reacción y la captura de Grafana.
7:30–8:30	**Costos**	Clúster: USD 0.34 por hora en reposo y 0.74 en el pico, **USD 0.003 por jugador y hora**. Bots: USD 0.60 por hora para 2,000. Espectadores: 20,000 son 54 TB por hora. Tabla de líderes: ≈ USD 6–7 al mes contra 15–20 de una VM.
8:30–9:30	**Lo que aprendimos**	Las fallas que encontraron las pruebas: `/ready` una sola vez, `SIGTERM` ignorado, EKS sin metrics-server, el límite de 10 Lambdas y el orden del `destroy`. La GPU fue 68 % más lenta que la CPU para un bot pequeño.
9:30–10:00	**Cierre**	Corremos `terraform destroy`: "todo lo que vieron se vuelve a crear con un comando". Preguntas.

Demo en vivo (3 minutos)

Antes de la clase dejamos listo el clúster con la flota, Grafana abierto en el tablero y Locust preparado.

1. Grafana en pantalla completa con los paneles de jugadores conectados, servidores por estado y nodos.
2. En la terminal enseñamos `kubectl get gameservers` y `kubectl get nodes`.

3. Arrancamos una carga corta para que se vea el escalado en 3 minutos: `locust -f locustfile.py Jugador -u 300 -r 15 -t 3m --headless`.
4. Mientras sube, explicamos: los jugadores entran, los servidores `Ready` pasan a `Allocated`, el FleetAutoscaler crea más para mantener servidores libres, y cuando ya no caben, el Cluster Autoscaler agrega un nodo.
5. Repetimos los dos `kubectl get` para enseñar los servidores y nodos nuevos.
6. Si un nodo tarda más de lo que dura la demo, enseñamos la captura de la rampa de 300 jugadores como respaldo y lo decimos.

Diapositivas

1. Título y equipo.
2. El problema.
3. El juego (captura).
4. Arquitectura (diagrama).
5. Las tres capas de escalado.
6. Demo (solo el título; cambiamos a Grafana).
7. Resultados (2 gráficas).
8. Costos (tabla).
9. Lo que aprendimos y recomendaciones.